Извјештај са лабораторијске вјежбе

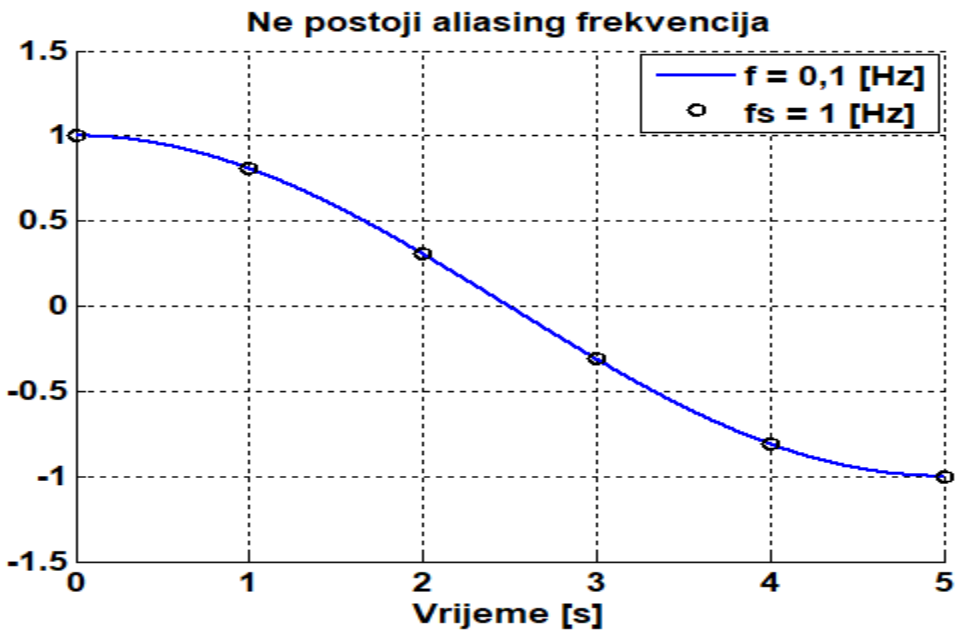
1. Фреквенцијско преклапање (*aliasing*):

Континуални сигнал који има Фуријеову трансформацију једнаку нули изван интервала $(-\omega_0, \omega_0)$ биће одређен на јединствен начин својим вриједностима у еквидистантним тачкама ако је фреквенција одабирања $\omega_s > 2 \cdot \omega_0$. Ако би фреквенција одабирања (ω_s) била таква да не задовољава услов $\omega_s > 2 \cdot \omega_0$, тада се, након дискретизације сигнала, одређена фреквенција не би могла разликовати од својих алиаса. Ова појава ће се десити ако за сигнал кружне учестаности ω_l и учестаности одабирања ω_s важи услов $\omega_l > \omega_s / 2$. Тада је основни алиас дат са:

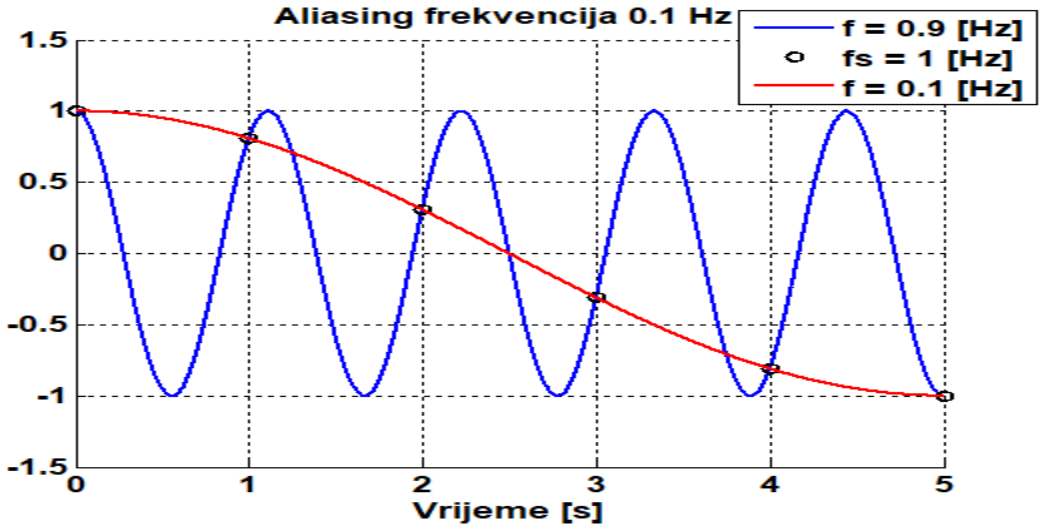
$$\omega = |(\omega_l + \omega_s / 2) \cdot \mathrm{mod}(\omega_s) - \omega_s / 2|.$$

Примјере из књиге добијамо у случајевима када је:

$$f = 0.1\ [Hz],\ f_s = 1\ [Hz] \text{ и } f = 0.9\ [Hz],\ f_s = 1\ [Hz].$$



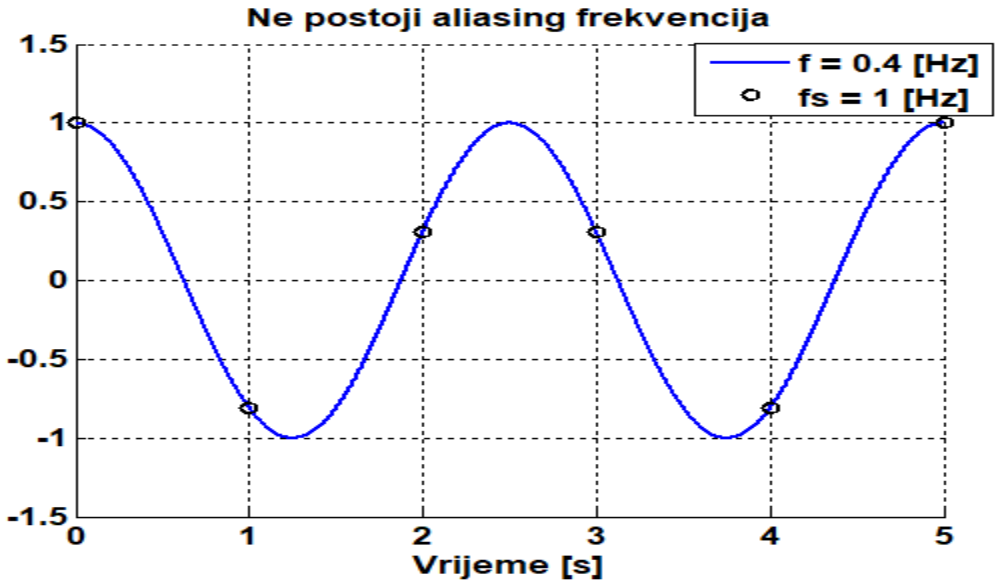
Сл. 1-1: Фреквенција континуалног сигнала f = 0.1 [Hz], фреквенција одабирања f_s = 1 [Hz]



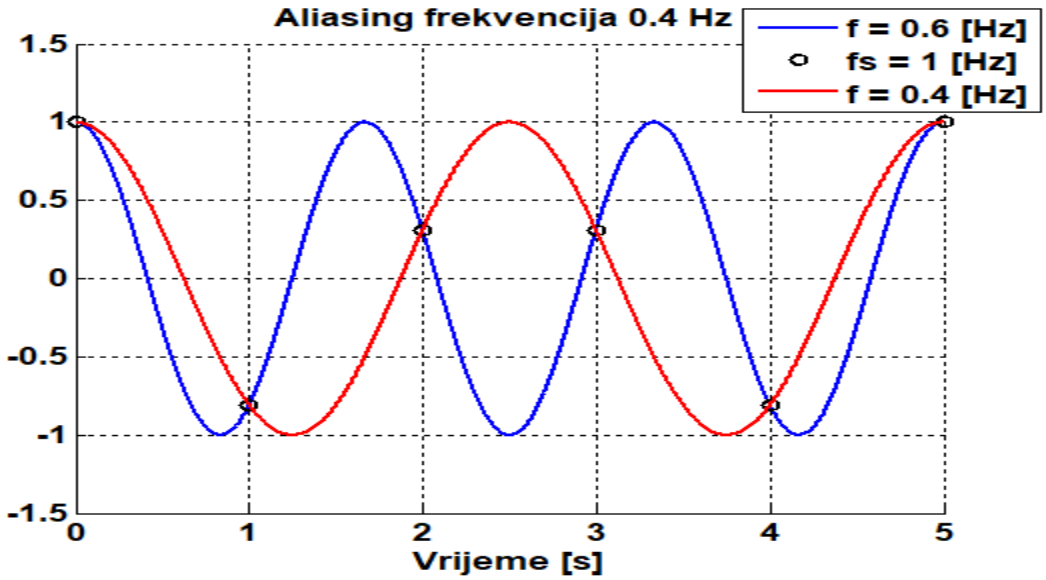
Сл. 1-2: Фреквенција континуалног сигнала f = 0.9 [Hz], фреквенција одабирања f_s = 1 [Hz]

Са *Сл. 1-1* видимо да не долази до појаве *aliasing* – а, јер је $f_s > 2 \cdot f$, док са *Сл. 1*-2 видимо да долази до појаве *aliasing* – а, јер је $f_s < 2 \cdot f$. На основу наведених примјера можемо закључити да се појава *aliasing* – а може избјећи у случају када континуални сигнал бирамо са бар два пута већом фреквенцијом. Овај закључак може се видјети и на сљедећим примјерима:

$$f = 0.4\ [Hz],\ f_s = 1\ [Hz] \text{ и } f = 0.6\ [Hz],\ f_s = 1\ [Hz].$$



Сл. 1-3: Фреквенција континуалног сигнала f = 0.4 [Hz], фреквенција одабирања f_s = 1 [Hz]

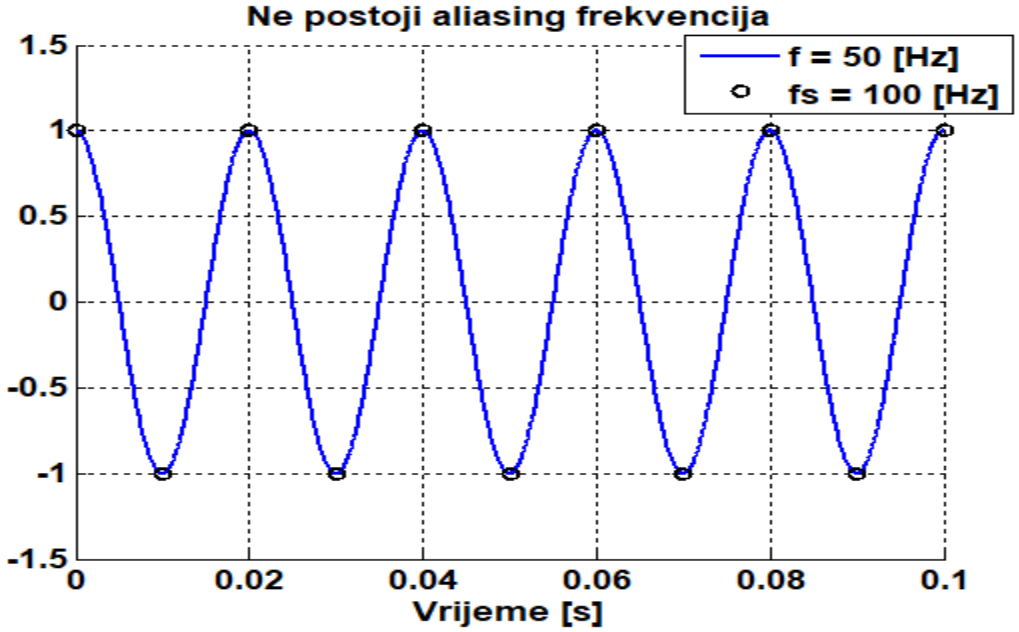


Сл. 1-4: Фреквенција континуалног сигнала f = 0.6 [Hz], фреквенција одабирања f_s = 1 [Hz]

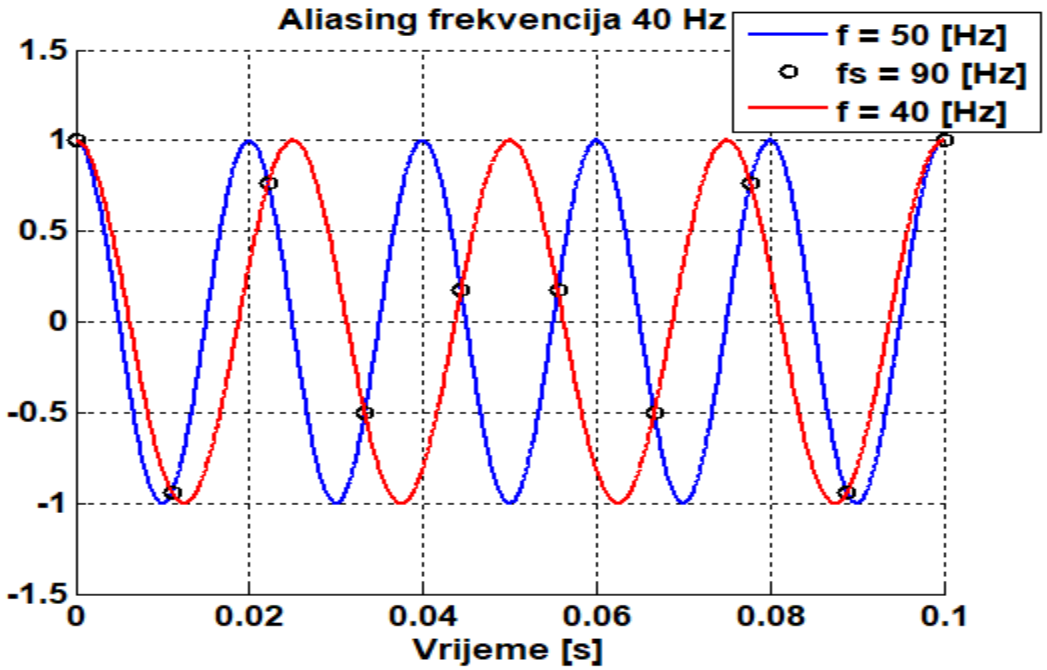
Узмимо за примјер још и случај да је фреквенција сигнала једнака фреквенцији електроенергетског система у Европи $f = 50$ [*Hz*]. До појаве *aliasing* – а ће доћи у случају ако је фреквенција одабирања мања од $f_s = 100$ [*Hz*], што се може видјети и у сљедећим примјерима:

$$f = 50 \text{ [Hz]}, \; f_s = 100 \text{ [Hz]} \text{ и } f = 50 \text{ [Hz]}, \; f_s = 90 \text{ [Hz]}.$$

С обзиром на претходно наведене вриједности фреквенције, у *Matlab* скрипти је потребно смањити вриједност времена трајања сигнала на *tMax = 0.1* [*s*].



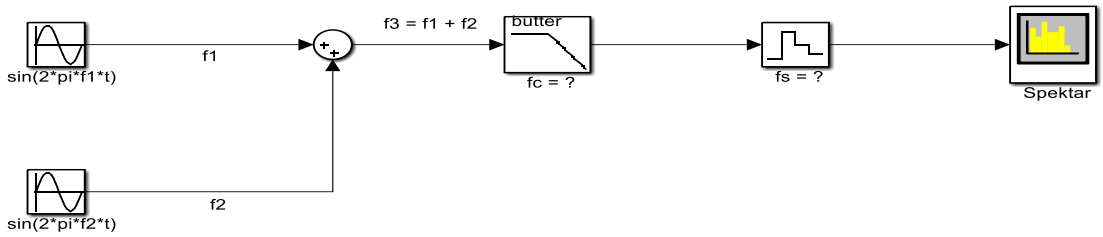
Сл. 1-5: Фреквенција континуалног сигнала f = 50 [Hz], фреквенција одабирања f_s = 100 [Hz]



Сл. 1-6: Фреквенција континуалног сигнала f = 50 [Hz], фреквенција одабирања f_s = 90 [Hz]

2. Предфилтри (*antialiasing* филтри):

У реалним ситуацијама појављују се проблеми због тога што сигнали који се користе у дигиталној обради често имају спектар који није једнак нули за фреквенције које су веће од Никвистове фреквенције, односно за које нису задовољени услови теореме одабирања. Због тога се прије A/D конверзије користе филтри који отклањају компоненте сигнала на високим фреквенцијама.

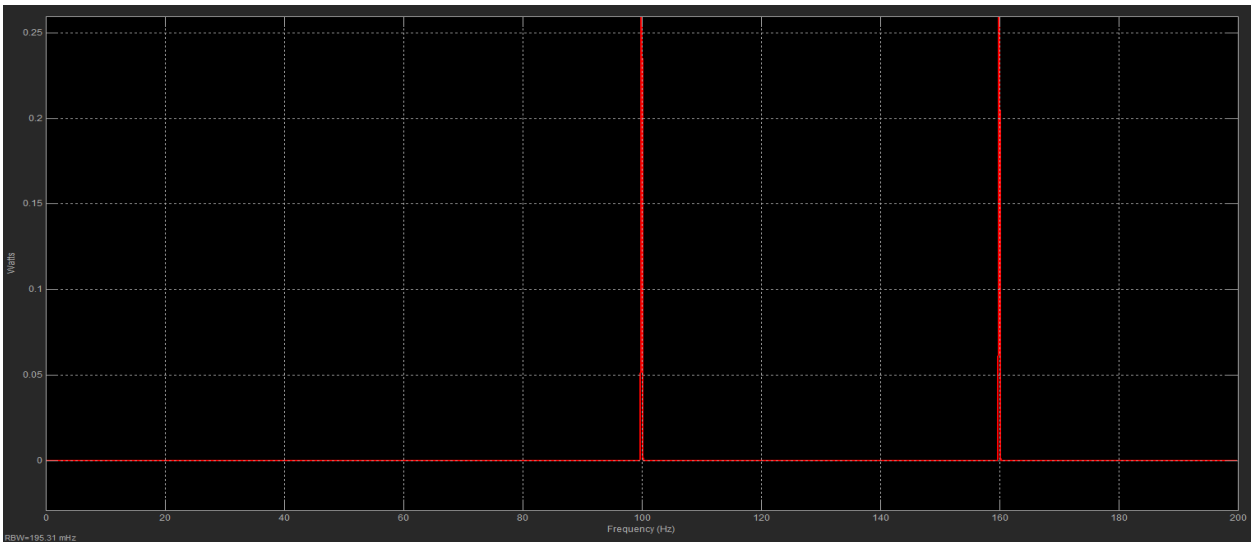


Сл. 2-1: Simulink модел који је коришћен приликом анализе

Да би се сигнал f_3 пропустио у цјелости потребно је да фреквенција филтра буде већа од највеће фреквенције у спектру. Пропуштени сигнал је потребно одабрати са бар два пута већом фреквенцијом од фреквенције спектра пропуштеног сигнала. На основу наведеног, одабране су сљедеће вриједности граничне фреквенције f_c и фреквенције одабирања f_s:

$$f_c = 200\ [Hz],\ \ f_s = 400\ [Hz].$$

Илустрација претходно наведеног је приказана на *Сл. 2-2*.



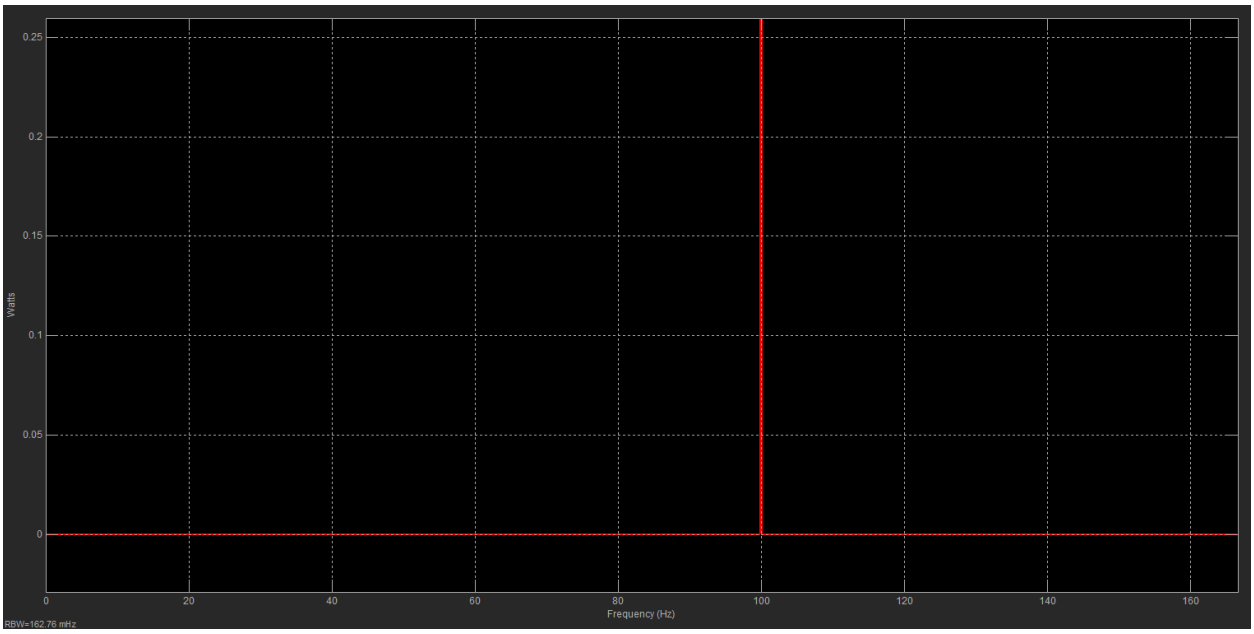
Сл. 2-2: Spectrum Analyzer – $f_1 = 100$ [Hz], $f_2 = 160$ [Hz], $f_c = 200$ [Hz], $f_s = 400$ [Hz]

Због лакшег очитавања графика са *Сл. 2-2*, *y* – оса изражена је у *Watts*. Када се покрене симулација у *Spectrum Analyzer* – у потребно је подесити *y* – осу за очитавање у *Watts* - у на сљедећи начин: *View* → *Spectrum Settings* → *Trace options* и у пољу *Units* одабрати *Watts*. Са *Сл. 2-2* видимо да филтар пропушта сигнале чије су фреквенције $f_1 = 100$ [*Hz*] и $f_2 = 160$ [*Hz*], јер су оне мање од фреквенције филтра која износи $f_c = 200$ [*Hz*].

Да би се пропустио само сигнал чија је фреквенција $f_1 = 100$ [*Hz*] потребно је да гранична фреквенција филтра буде веће од *100* [*Hz*], а мања од *160* [*Hz*]. На основу наведеног, одабране су сљедеће вриједности граничне фреквенције f_c и фреквенције одабирања f_s:

$$f_c = 105 \text{ [Hz]}, \ f_s = 333{,}33 \text{ [Hz]}.$$

Илустрација претходно наведеног је приказана на *Сл. 2-3*.

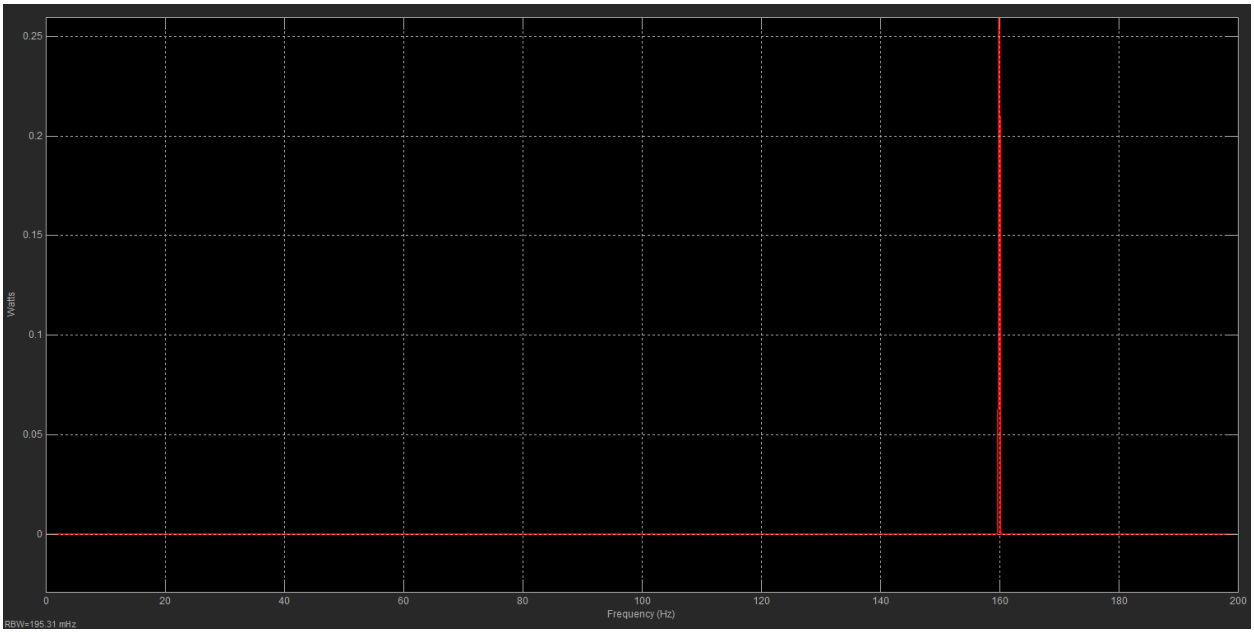


Сл. 2-3: Spectrum Analyzer – $f_1 = 100$ [*Hz*], $f_c = 105$ [*Hz*], $f_s = 333{,}33$ [*Hz*]

Да би се пропустио само сигнал чија је фреквенција $f_2 = 160$ [*Hz*] потребно је одабрати *Bandpass* тип филтра. У том случају потребно је одабрати двије граничне фреквенције, при чему једна мора бити између фреквенција сигнала спектра $f_1 = 100$ [*Hz*] и $f_2 = 160$ [*Hz*], а друга мора бити већа од $f_2 = 160$ [*Hz*]. На основу наведеног, одабране су сљедеће вриједности граничних фреквенција f_{c1} и f_{c2} и фреквенције одабирања f_s:

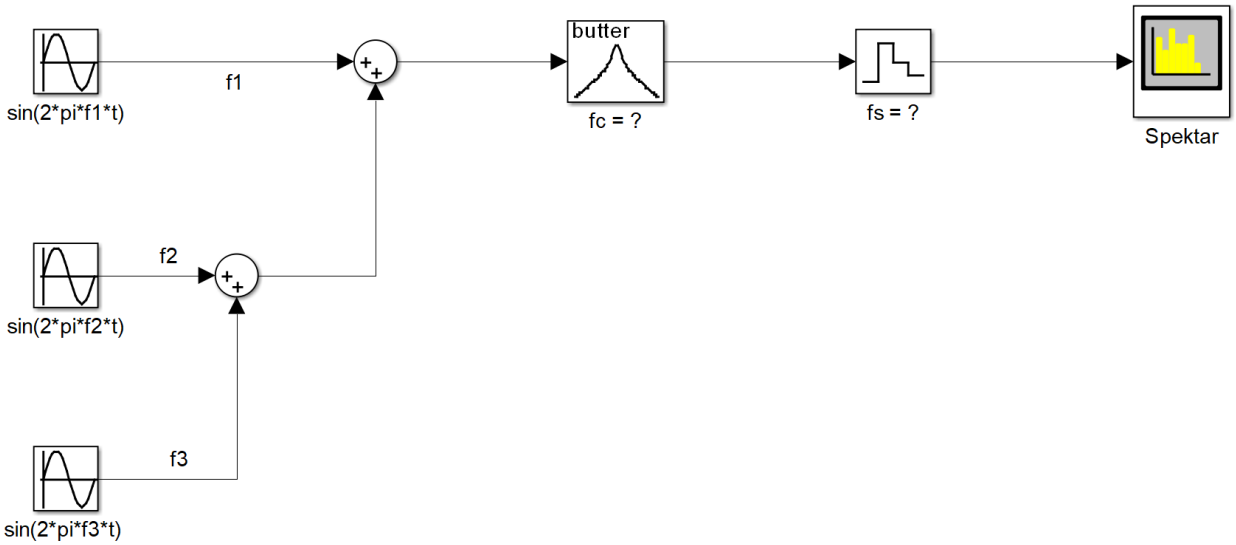
$$f_{c1} = 130 \text{ [Hz]}, \ f_{c2} = 180 \text{ [Hz]}, \ f_s = 400 \text{ [Hz]}.$$

Илустрација претходно наведеног је приказана на *Сл. 2-4*.



Сл. 2-4: Spectrum Analyzer – $f_2 = 160$ [*Hz*], $f_{c1} = 130$ [*Hz*], $f_{c2} = 180$ [*Hz*], $f_s = 400$ [*Hz*]

Ако се на један улазни сигнал, нпр. $f_2 = 160$ [*Hz*] дода и шум (високофреквенцијска компонента) *Simulink* модел ће изгледати као што је приказано на сљедећој слици.



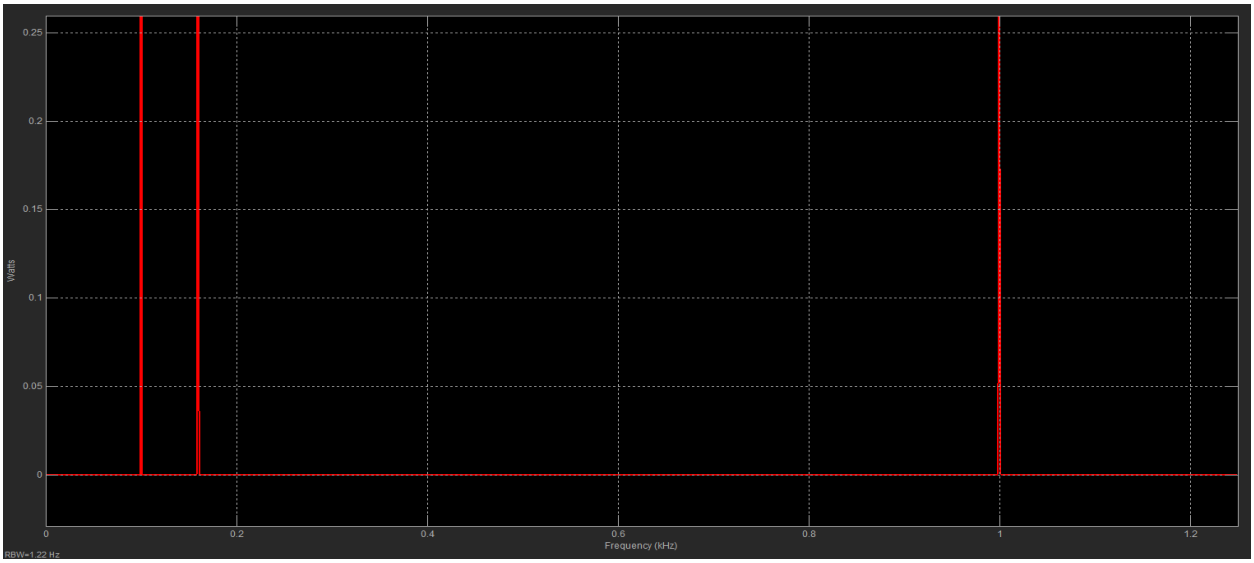
Сл. 2-5: Simulink модел који је коришћен приликом анализе када се на један улазни сигнал дода шум

Нека се шум појавио на фреквенцији *1000* [*Hz*]. Тада је потребно изабрати одговарајућу граничну фреквенцију филтра f_c и фреквенцију одабирања f_s да би се на *Spectrum Analyzer* – у добио изворни сигнал. То значи да спектар мора имати три компоненте сигнала чије су фреквенције *100* [*Hz*], *160* [*Hz*] и *1000* [*Hz*], респективно. Да

би филтар пропустио све компоненте сигнала, његова гранична фреквенција мора бити већа од *1000* [*Hz*]. На основу наведеног, одабране су сљедеће вриједности граничне фреквенције f_c и фреквенције одабирања f_s:

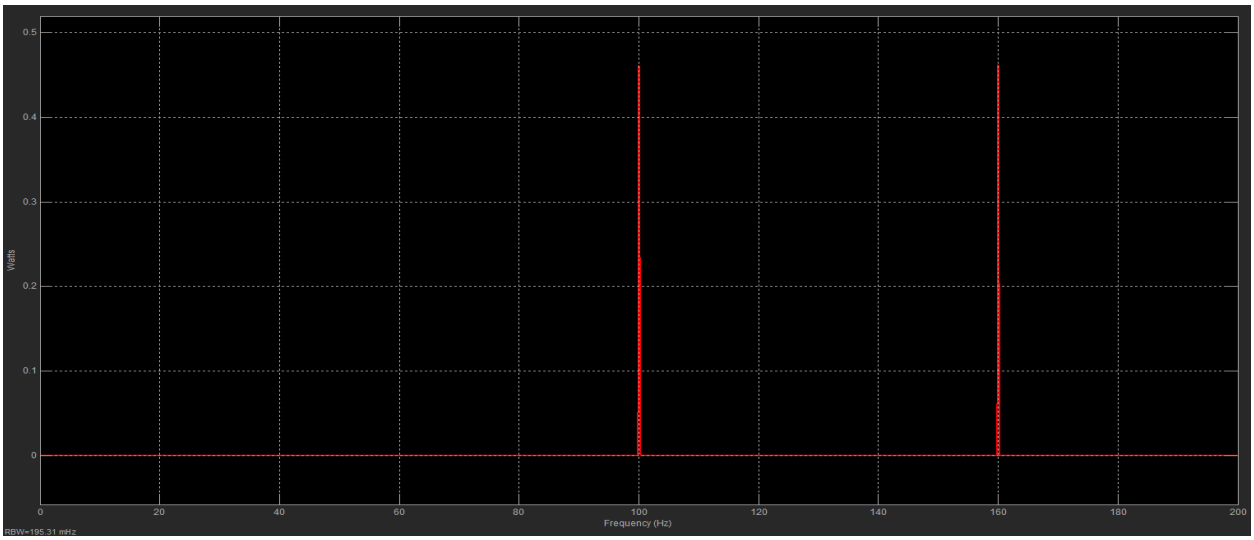
$$f_c = 1250\ [Hz],\ \ f_s = 2500\ [Hz].$$

Илустрација претходно наведеног је приказана на *Сл. 2-6*.

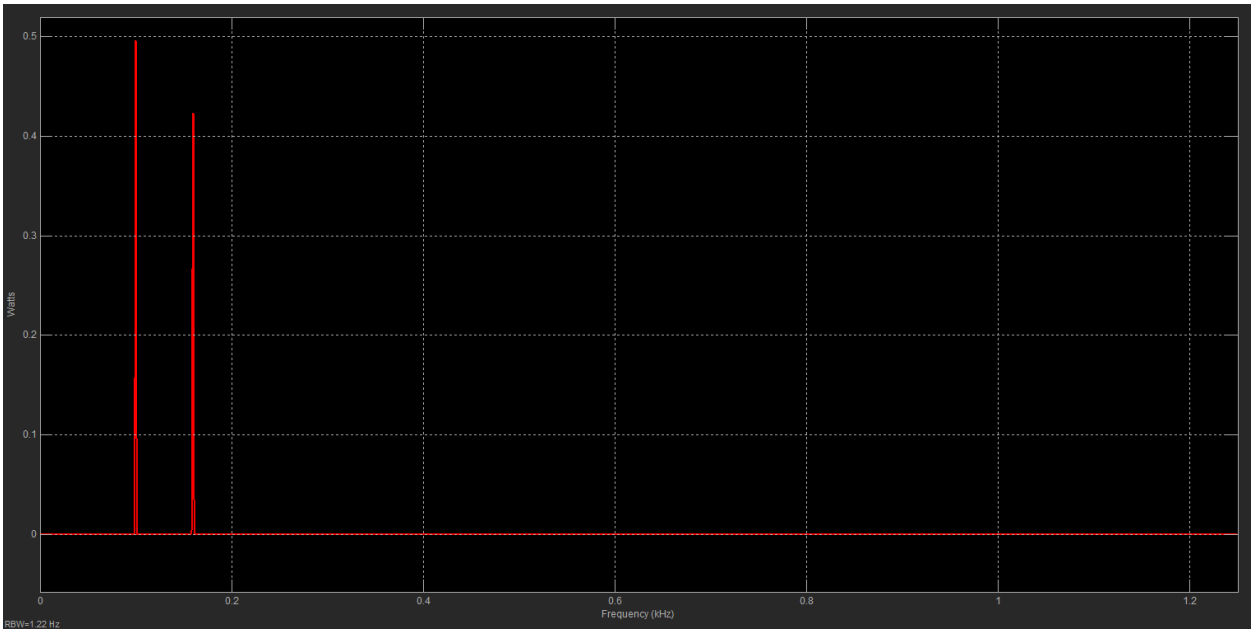


Сл. 2-6: Spectrum Analyzer - $f_1 = 100$ [*Hz*], $f_2 = 160$ [*Hz*], $f_3 = 1000$ [*Hz*], $f_c = 1250$ [*Hz*], $f_s = 2500$ [*Hz*]

Ако анализатор спектра не треба да види шум, онда га је потребно уклонити помоћу филтра. У том случају потребно је одабрати граничну фреквенцију филтра већу од *160* [*Hz*] и фреквенцију одабирања бар два пута већу од *160* [*Hz*].



Сл. 2-7: Spectrum Analyzer - $f_1 = 100$ [*Hz*], $f_2 = 160$ [*Hz*], $f_3 = 1000$ [*Hz*], $f_c = 200$ [*Hz*], $f_s = 400$ [*Hz*]



Сл. 2-8: Spectrum Analyzer - $f_1 = 100$ [*Hz*], $f_2 = 160$ [*Hz*], $f_3 = 1000$ [*Hz*], $f_c = 200$ [*Hz*], $f_s = 2500$ [*Hz*]

На основу резултата који су приказани на *Сл. 2-7* и *Сл. 2-8* закључујемо да након елиминације шума, сигнал можемо одабрати са фреквенцијом одабирања $f_s = 400$ [*Hz*] умјесто са $f_s = 2500$ [*Hz*].

3. Z, Z^{-1}, Z_m трансформација:

Прво ћемо илустровати примјену функција *"ilaplace", "ztrans", "iztrans"* у програмском пакету *Matlab*.

```
>> syms s t z
>> F_s(s) = 1/(s*(s+1))

F_s(s) =

1/(s*(s + 1))

>> f_t(t) = ilaplace(F_s)

f_t(t) =

1 - exp(-t)

>> F_z(z) = ztrans(f_t)

F_z(z) =

z/(z - 1) - z/(z - exp(-1))

>> f_t2(t) = iztrans(F_z)

f_t2(t) =

exp(-1)*(exp(1)*kroneckerDelta(n, 0) - exp(-1)^n*exp(1)) - kroneckerDelta(n, 0) + 1
```

Сл. 3-1: Примјер примјене функција "ilaplace", "ztrans", "iztrans"

У *Matlab Help* – у може се наћи објашњење за функцију *kroneckerDelta*. Прототип ове функције је **kroneckerDelta(m, <n>)**. Ова функција враћа вриједност 1 ако су **m** и **n** једнаке, а враћа вриједност 0 ако су **m** и **n** различите вриједности.

Сада ћемо показати како се врши дискретизација система описаног моделом у простору стања у програмском пакету *Matlab*. Ово ћемо илустровати на једном од примјера који су урађени на аудиторним вјежбама. *Matlab* код који илуструје овај примјер приказан је на *Сл. 3-2*.

```
>> A = [0 1; -0.5 -1.5];
>> B = [0; 1];
>> C = [2 1];
>> D = 0;
>> Sys_c = ss(A,B,C,D)

Sys_c =

  a =
          x1    x2
   x1      0     1
   x2   -0.5  -1.5

  b =
       u1
   x1   0
   x2   1

  c =
       x1  x2
   y1   2   1

  d =
       u1
   y1   0

Continuous-time state-space model.

>> T = 0.5;
>> Sys_d = c2d(Sys_c, T)
```

Сл. 3-2: Matlab код

Рјешење задатка у *Matlab* – у, тј. дискретни модел у простору стања приказан је на *Сл. 3-3*.

```
Sys_d =

  a =
             x1       x2
   x1    0.9511   0.3445
   x2   -0.1723   0.4343

  b =
             u1
   x1   0.09786
   x2    0.3445

  c =
       x1  x2
   y1   2   1

  d =
       u1
   y1   0

Sample time: 0.5 seconds
Discrete-time state-space model.
```

Сл. 3-2: Дискретни модел система у простору стања

Ако упоредимо дискретни модел система у простору стања који смо добили у програмском пакету *Matlab* са моделом који смо добили на аудиторним вјежбама, можемо закључити да су резултати идентични.